



Frameworks WEB

Proteção de senhas com HASH e SALT

Índice

Criptografando senhas.....	3
Utilização de HASH.....	3
Aumentando a segurança com SALT.....	4
Atualizando o BD para uso de HASH e SALT.....	5
Modificando a API.....	6
Testes finais.....	7
Histórico de revisões.....	8



Criptografando senhas

Utilização de HASH

Na aula passada conseguimos configurar nosso sistema para liberar o acesso a um grupo de usuários cadastrados em um Banco de Dados. Ocorre que na ocasião, as senhas dos usuários também foram armazenadas explicitamente no banco. É muito importante entender que isso não é uma prática segura, pois qualquer pessoa que tenha acesso a esses dados, conseguiria facilmente identificar as senhas dos usuários.

Para resolver esse problema podemos fazer uso de um algoritmo unidirecional para geração de uma sequência de dados (HASH) a partir de uma determinada entrada (por exemplo, uma senha). Por ser unidirecional, o algoritmo não permite que possamos descobrir a senha original a partir da sequência de dados gerada. Podemos armazenar essa sequência no banco de dados e futuramente compará-la com a sequência gerada pelo algoritmo a partir de novos logins do usuário.

Segue uma explicação muito legal sobre esses tipos de algoritmos:

<https://www.voitto.com.br/blog/artigo/o-que-e-hash-e-como-funciona>

Em nosso sistema vamos utilizar um algoritmo chamado de SHA256. Seguem alguns links de sites que executam esse algoritmo de forma online:

<https://xorbin.com/tools/sha256-hash-calculator>

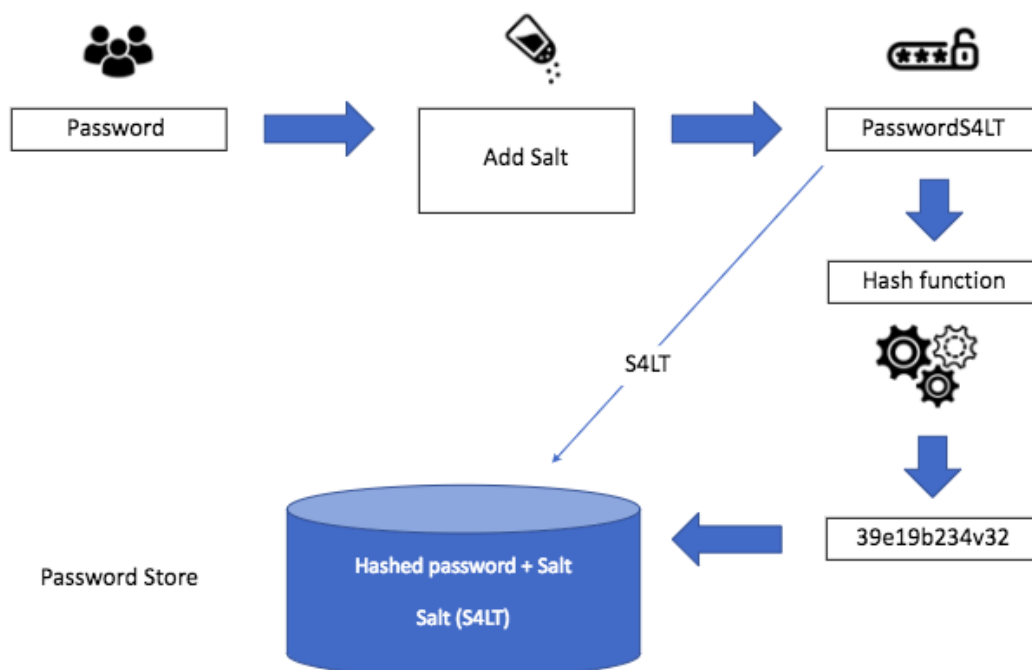
<https://codebeautify.org/sha256-hash-generator>

Experimente entrar com uma provável senha na entrada do algoritmo e verifique qual será a sequência de dados (HASH) gerada.

Aumentando a segurança com SALT

Infelizmente ainda é muito comum existirem pessoas que utilizem senhas extremamente previsíveis (<https://www.penso.com.br/conheca-as-senhas-mais-hackeadas-no-mundo-e-saiba-como-criar-uma-senha-segura/>), permitindo assim identificar sequências de HASH idênticas que facilitariam o processo de descoberta de uma senha.

Para contornar esse problema, podemos acrescentar um trecho aleatório de caracteres (SALT) à senha, evitando que senhas idênticas acabem gerando o mesmo HASH. Dessa forma, o SALT deve também ser armazenado no banco de dados para permitir realizar a validação futura da senha do usuário. Segue um diagrama que demonstra essa operação:



<https://www.okta.com/blog/2019/03/what-are-salted-passwords-and-password-hashing/>



Atualizando o BD para uso de HASH e SALT

Substitua o campo "senha" dos usuários cadastrados no seu Banco de Dados por dois campos chamados de "hash" e "salt". Para fazermos um primeiro teste, defina o "salt" de um usuário com o valor "\$AB%G6"

The screenshot shows the MongoDB Compass interface. On the left, the 'tasksDB' database is selected, and the 'users' collection is highlighted. The main panel shows a filter bar with the text '{ field: 'value' }'. Below the filter, it says 'QUERY RESULTS 1-1 OF 1'. A single document is displayed with the following fields: `_id: ObjectId("627e5093cb07dd5d809971d9")`, `nome: "Maria"`, `hash: ""`, and `salt: "$AB%G6"`.

Utilize um site de geração de HASH SHA256, utilizando a senha "1234" acrescentada do SALT acima, ou seja: "1234\$AB%G6". Grave o HASH obtido no campo "hash" do usuário cadastrado no banco. Ex:

The screenshot shows the MongoDB Compass interface. On the left, the 'tasksDB' database is selected, and the 'users' collection is highlighted. The main panel shows a filter bar with the text '{ field: 'value' }'. Below the filter, it says 'QUERY RESULTS 1-1 OF 1'. A single document is displayed with the following fields: `_id: ObjectId("627e5093cb07dd5d809971d9")`, `nome: "Maria"`, `hash: "ce62f8015923e98b0a4445abf4c9de77cb12b79201573b9b3a5af7ab8bb9a159"`, and `salt: "$AB%G6"`.

Pronto, agora a senha dos seus usuários estarão melhor protegidas.

Modificando a API

Agora vamos modificar nossa API para realizar a autenticação de usuários considerando o uso de HASH+SALT.

Modifique o arquivo "**user.js**", substituindo o campo "**senha**" pelos campos "**hash**" e "**salt**". Ex:

```
var mongoose = require('mongoose');

var userSchema = new mongoose.Schema({
  nome: {
    unique: true,
    type: String
  },
  hash: String,
  salt: String
},
{
  versionKey: false
});

module.exports = mongoose.model('User', userSchema)
```

Insira o seguinte método para validação de senha ao final de "**routes.js**":

```
var { createHash } = require('crypto');
function validPassword (senha, hashBD, saltBD) {
  hashCalculado=createHash('sha256').update(senha+saltBD).digest('hex');
  return hashCalculado === hashBD;
};
```



Substituir a seguinte linha de validação da senha:

```
if (data != null && data.senha === req.body.senha) {
```

existente dentro do tratamento da rota de login, por:

```
if (data!=null && validPassword(req.body.senha, data.hash, data.salt)) {
```

Testes finais

Testar a nova autenticação através do POSTMAN e em seguida utilizando o APP TODO.

Histórico de revisões

Revisão: 01
Data: 16/05/2023
Descrição das alterações:
 Adaptação para curso presencial

Revisão: 00
Data: 13/05/2022
Descrição das alterações:
 Documento original